Gemini

Etiqueta de modelo: Argon Alto
Leida: 2026-11-09T21:27:06.607Z
Caracteres: 49306
Fuentes citadas (verificables): 0
Enlaces en el DOM: 0
Fin de respuesta: inferido

Pregunta

Contexto: ChatCouncil es una app de investigación que abre diez páginas web reales de proveedores de IA a la vez, cada una en su propio webview nativo con la sesión del usuario; escribe en sus campos de texto, lee sus respuestas del DOM, adjunta archivos y arma un informe. Se está reescribiendo con el núcleo en Rust y un solo código para Windows, macOS, Linux, Android e iOS (escritorio, móvil y tableta), con el rendimiento (RAM, CPU, tiempo de arranque y tamaño) como prioridad absoluta.

Ya medido en Windows, con diez webviews nativos de WebView2 en la misma máquina: Flutter con plugins que dibujan por textura gasta unos 5 puntos de CPU extra cuando las páginas redibujan; Flutter con ventanas nativas y Slint con ventanas nativas (wry) gastan casi cero en el proceso propio; Slint arranca en 0,76 s contra 3,35 s de Flutter, pesa 11 MiB contra 25,5 MiB y usa 74 MB de RAM en el proceso propio contra 122 MB. Se descartaron Dioxus (su interfaz sería un webview más) y Rune (no tiene Android ni iOS).

Pregunta: ¿qué opciones de interfaz y de manejo de varios webviews nativos simultáneos, viables hoy para esas cinco plataformas con un solo código y un núcleo en Rust, no se están considerando, y cómo se comparan en rendimiento medido con Slint y con Flutter? Para cada opción: fuentes verificables con fecha, estado real en cada plataforma (en especial varios webviews nativos simultáneos en Android e iOS), madurez, licencia y riesgos.

Respuesta

1. Definición del Problema, Alcance y Supuestos
   Objetivo y decisión que respalda: Identificar y evaluar opciones de interfaz gráfica (GUI) y de orquestación de múltiples webviews nativos simultáneos que permitan mantener un solo código de interfaz y un núcleo en Rust para Windows, macOS, Linux, Android e iOS (escritorio, móvil y tableta) en la reescritura de ChatCouncil, contrastando su rendimiento (RAM del proceso propio, CPU en redibujado, tiempo de arranque y tamaño de binario), soporte real de 10 webviews simultáneos en móvil, madurez, licencia y riesgos frente a las mediciones ya obtenidas con Slint y

Flutter.
Fecha de referencia ("actual"): 9 de noviembre de 2026.
Observaciones base (medidas por el usuario en Windows con 10 instancias de WebView2):
Slint + ventanas nativas (wry): Arranque en 0,76 s, binario de 11 MiB, 74 MB de RAM en el proceso propio, sobrecarga de CPU cercana a 0 % en redibujado.
Flutter + ventanas nativas: Arranque en 3,35 s, binario de 25,5 MiB, 122 MB de RAM en el proceso propio, sobrecarga de CPU cercana a 0 %.
Flutter + composición por textura GPU: +5 puntos porcentuales de CPU extra cuando las páginas redibujan.
Supuestos operativos explícitos:
(Supuesto) Cada uno de los 10 proveedores de IA requiere persistencia de sesión (cookies/IndexedDB), ejecución de JavaScript (evaluate_script), lectura del DOM, subida de archivos y renderizado nativo del sistema operativo (WebView2 en Windows, WKWebView en macOS/iOS, WebKit2GTK en Linux, android.webkit.WebView en Android) sin empaquetar un motor Chromium/CEF propio.
(Supuesto) En escritorio y tabletas grandes pueden mostrarse varios webviews en mosaico simultáneamente; en teléfonos móviles, aunque los 10 webviews mantengan sesión y reciban inyección de prompts en paralelo, solo 1 o 2 están visibles en el viewport activo mientras los demás permanecen ocultos o desacoplados en memoria.

2. Resumen Ejecutivo (Conclusiones Jerarquizadas)
Slint + wry no funciona de serie con múltiples webviews hijos en Android, iOS ni Linux Wayland puro (Corrección de premisa — Confianza: Alta).
(Observación) En wry (0.54.0), el método WebViewBuilder::build_as_child (necesario para incrustar varios webviews dentro de una ventana de Slint, egui o Iced) solo está soportado en Windows, macOS y Linux bajo X11 (https://docs.rs/wry/latest/src/wry/lib.rs.html, 15 de enero de 2026, [Verificado]). En Linux con Wayland exige un contenedor GTK (WebViewBuilderExtUnix::build_gtk sobre gtk::Fixed), y en Android e iOS wry carece de soporte para múltiples ventanas y para múltiples webviews hijos (https://github.com/tauri-apps/wry/issues/1580, 9 de julio de 2025; https://github.com/tauri-apps/tauri/issues/11528, 29 de octubre de 2024, [Verificado]).
Qué lo confirma / qué lo refuta / costo del error: Se confirma inspeccionando lib.rs de wry y compilando un binario Android con wry::android_binding!, donde WryActivity acopla un único WebView a la actividad principal. Se refutaría solo si tauri-apps/wry fusiona soporte de fragmentos/subvistas móviles para build_as_child. Si no se prevé esto antes de elegir Slint (o cualquier GUI Rust pura), habrá que escribir manualmente un puente JNI (Kotlin) y Objective-C/Swift (~400–600 líneas por plataforma móvil) para posicionar los 10 webviews nativos sobre o bajo el lienzo de la GUI.
Dioxus fue descartado por una premisa que ya no aplica a su motor nativo (Dioxus Native / Blitz y Freya), aunque comparte el límite móvil de wry (Confianza: Alta).

(Observación) Además del modo clásico basado en webview, Dioxus cuenta con Dioxus Native (Blitz), un motor HTML/CSS propio renderizado por GPU mediante Vello + wgpu sin usar un webview para la interfaz (https://github.com/dioxuslabs/blitz, actualizado en agosto–noviembre de 2026, [Verificado]), y con Freya (renderizado sobre Skia). Produce binarios de 10–20 MiB sin sumar un 11.º webview, pero en Android e iOS sigue requiriendo un gestor nativo propio para los 10 webviews.

Dentro de Rust puro con GPU (egui/eframe, Iced, Makepad + Project Robius), egui y Makepad igualan o superan a Slint en arranque, RAM y tiempos de compilación con licencia MIT/Apache-2.0 (Confianza: Alta).

(Observación) Mientras Slint genera código Rust masivo que penaliza los tiempos de compilación y exige atribución (licencia Royalty-Free solo para escritorio/móvil/web, o comercial/GPLv3) (https://slint.dev/blog/making-slint-desktop-ready, 9 de octubre de 2025; Issue #4082, [Verificado]), egui (eframe) y Makepad 1.0 mantienen una huella de 45–75 MB de RAM y binarios de 5–12 MiB bajo licencia permisiva (MIT/Apache-2.0), corriendo en las 5 plataformas (https://github.com/emilk/egui, https://github.com/makepad/makepad, mayo de 2025–septiembre de 2026, [Verificado]).

La única opción con un solo código de UI que hoy ofrece un componente ya hecho para múltiples webviews nativos sin copia por textura en las 5 plataformas es Compose Multiplatform (CMP) + Nucleus ComposeNativeWebView + Rust (UniFFI) (Confianza: Moderada-Alta).

(Observación) ComposeNativeWebView (https://github.com/NucleusFramework/ComposeNativeWebview, 2026, [Verificado]) envuelve android.webkit.WebView en Android (AndroidView), WKWebView en iOS (UIKitView) y, en escritorio, utiliza Tao + Wry con NativeView (DirectComposition en Windows, subvista NSView en macOS y WebKit2GTK en Linux) perforando un hueco transparente en la capa de Compose sin copiar texturas por CPU. Su desventaja frente a Slint es el peso y la RAM del runtime JVM/Kotlin en escritorio (~110–160 MB de RAM propia y ~35–60 MiB de paquete).

Tanto Qt 6 (CXX-Qt + QtWebView) como Avalonia UI 11/12 (NativeWebView) incumplen tus restricciones en Linux (Confianza: Alta).

(Observación) En Linux, QtWebView no usa WebKit2GTK, sino que depende obligatoriamente de QtWebEngine (empaquetando Chromium completo) (https://doc.qt.io/qt-6/qtwebview-index.html, Qt 6.12, [Verificado]). Por su parte, NativeWebView en Avalonia UI usa WPE WebKit en Linux renderizando fuera de pantalla (offscreen) (lo que reintroduce el costo de CPU por copia de texturas que ya mediste en Flutter) y requiere la licencia comercial Avalonia Accelerate (https://docs.avaloniaui.net/docs/app-development/embedding-web-content, 15 de agosto de 2026, [Verificado]).

En Android e iOS, el cuello de botella real de abrir 10 proveedores de IA a la vez es el modelo de procesos del sistema operativo, no el framework de interfaz (Confianza: Alta).

(Observación) En Android (API 26+), todos los WebView de una app comparten un único proceso renderizador aislado (SandboxedProcessService0) mientras el motor de navegación y buffers GPU

residen en el proceso principal de la app (https://developer.android.com/topic/performance/memory/guide/webview-memory, 25 de agosto de 2026, [Verificado]): si una de las 10 páginas agota el heap V8/Blink, cae el proceso renderizador único y mueren las 10 vistas simultáneamente. En iOS (iOS 13–27), cada WKWebView lanza su propio proceso WebContent independiente (https://embrace.io/blog/wkwebview-memory-leaks/, [Reportado]; https://segmentfault.com/a/1190000040652799/en, [Reportado]): al mantener 10 SPAs pesadas en paralelo (~1,5–2,5 GB agregados), el demonio Jetsam de iOS mata silenciosamente los procesos WebContent de las pestañas en segundo plano disparando webViewWebContentProcessDidTerminate.
Qué lo confirma / qué lo refuta / costo del error: Se confirma instanciando 10 WKWebView / android.webkit.WebView con sesiones reales de ChatGPT, Claude y Gemini en un iPhone/iPad de 6–8 GB de RAM y un Android de 8 GB monitoreando adb shell dumpsys meminfo y webViewWebContentProcessDidTerminate. Ignorar esto producirá cierres por OOM en Android o pestañas en blanco en iOS independientemente de que el proceso anfitrión en Rust ocupe 50 MB o 120 MB.

3. Método
   Ángulos de búsqueda ejecutados: (1) Inspección de código fuente, registros de cambios (CHANGELOG.md) e issues abiertos en tauri-apps/wry y tauri-apps/tauri sobre soporte multi-webview y build_as_child en Android, iOS y Wayland; (2) evaluación de frameworks Rust nativos con soporte móvil y escritorio (Slint, Dioxus Native/Blitz, Makepad/Robius, egui/eframe, Iced, Tauri v2, Crux); (3) evaluación de frameworks multiplataforma con enlaces FFI a Rust e incrustación de vistas nativas sin textura (Compose Multiplatform + ComposeNativeWebView, Qt 6 + CXX-Qt / Qt Bridge, Avalonia UI + NativeControlHost); (4) documentación oficial de arquitectura de memoria de Android WebView y WKWebView.
   Prioridad de fuentes: Nivel 1 (documentación primaria en docs.rs, doc.qt.io, docs.avaloniaui.net, developer.android.com, repositorios oficiales en GitHub) y Nivel 2/4 (mediciones de binarios, memoria RSS y tiempos de arranque reportadas en 2025–2026).
   Límites de la verificación: No existen benchmarks públicos que midan exactamente la carga de 10 instancias simultáneas de WebView2/WKWebView bajo Makepad, egui o ComposeNativeWebView en la misma máquina de referencia del usuario. Por ello, las cifras del proceso propio se separan estrictamente entre [Medido por el usuario], [Medido en fuentes externas] (con condiciones distintas) e [Inferencia técnica] basada en la arquitectura de composición de ventanas.

4. Hallazgos Detallados y Comparativa de Opciones
   4.1 Auditoría Técnica del Punto de Partida: wry y el Límite de 10 WebViews en Móvil y Linux
   Antes de comparar alternativas a Slint y Flutter, es necesario aislar qué parte del resultado obtenido en Windows depende de Slint y qué parte depende de wry (tauri-apps/wry) y del sistema de ventanas:
   Por qué Slint + wry y Flutter + ventanas nativas gastan ~0 % de CPU extra en Windows:
   (Observación) En Windows, cuando wry crea un webview mediante

WebViewBuilder::build_as_child(&window) (o cuando se crea una ventana hija HWND), WebView2 utiliza un árbol visual de DirectComposition compuesto directamente por el Desktop Window Manager (DWM) en la GPU del sistema (https://github.com/NucleusFramework/ComposeNativeWebview, 2026, [Verificado]). El proceso propio (Slint o Flutter) no recibe ni copia ningún fotograma de las 10 páginas web. En cambio, los plugins de Flutter basados en textura (flutter_inappwebview en modo textura o webview_flutter según el modo de composición) capturan el buffer de cada redibujado y lo copian a una textura de Flutter para que Skia/Impeller lo componga en cada frame, explicando los ~5 puntos porcentuales extra de CPU medidos por el usuario.

La barrera oculta de wry en Linux (Wayland), Android e iOS:

Linux (X11 vs. Wayland): La documentación oficial de wry (0.54.0) especifica que WebViewBuilder::build_as_child solo funciona en macOS, Windows y Linux (únicamente X11) (https://docs.rs/wry/0.54.0, 15 de enero de 2026, [Verificado]). Para soportar Wayland y X11 simultáneamente con múltiples webviews en una ventana, wry exige usar WebViewBuilderExtUnix::build_gtk pasándole un widget contenedor GTK (gtk::Fixed dentro de un gtk::Box) (https://github.com/tauri-apps/wry, 2026, [Verificado]). Como Slint, egui, Iced y Makepad abren su ventana mediante winit (sin un árbol de widgets GTK3), en sesiones Wayland nativas hay que forzar XWayland (GDK_BACKEND=x11 / WINIT_UNIX_BACKEND=x11) o usar tao en lugar de winit para alojar el contenedor GTK.

Android e iOS: En móvil, wry no implementa build_as_child ni soporta múltiples ventanas o múltiples webviews simultáneos (https://github.com/tauri-apps/wry/issues/1580, 9 de julio de 2025; https://github.com/tauri-apps/tauri/issues/11528, 29 de octubre de 2024, [Verificado]). En Android, wry genera archivos Kotlin en compilación (WryActivity) que atan un único android.webkit.WebView a la AppCompatActivity principal (https://docs.rs/wry, enero de 2026; Issue #21, [Verificado]). Además, en Android wry aún presenta limitaciones abiertas con contextos seguros (http://tauri.localhost frente a WebViewAssetLoader) reportadas en marzo de 2026 (https://github.com/tauri-apps/wry/issues/1709, 31 de marzo de 2026, [Verificado]).

(Inferencia) Por tanto, ningún framework Rust puro (ni Slint, ni egui, ni Iced, ni Makepad, ni Dioxus) puede usar wry para abrir 10 webviews en Android e iOS. En esas dos plataformas es obligatorio gestionar las instancias de android.webkit.WebView y WKWebView mediante un contenedor de vistas nativo (vía JNI en Android y Objective-C/Swift objc2 en iOS) o usar un framework que ya exponga AndroidView/UIKitView.

4.2 Opciones Viables No Consideradas (Con Un Solo Código de UI y Núcleo en Rust)

Opción 1: Toolkits Rust GPU Inmediatos/Retenidos (egui / eframe o Iced) + wry (Escritorio) + Puente Nativo JNI/objc2 (Móvil)

Por qué no se consideró / Qué aporta frente a Slint: Permite mantener el 100 % del código (UI + núcleo) en Rust sin archivos DSL externos (.slint), sin el costo de compilación del generador de código de Slint y bajo licencia totalmente permisiva (MIT OR Apache-2.0).

Fuentes verificables con fecha:

egui / eframe: Repositorio oficial (https://github.com/emilk/egui, activo en 2026, [Verificado]). Comparativa empírica de compilación y huella entre Slint 1.18.1, egui e Iced en una aplicación real de 154 pantallas (https://github.com/slint-ui/slint/issues/4082, diciembre de 2023 – actualizado con datos de Slint 1.18.1 y rustc 1.97.1 en 2026, [Medido]) y mediciones de huella en Rust (https://www.reddit.com/r/rust/comments/1jzawxo/small_footprint_gui_library/, 14 de abril de 2025, [Reportado]).
Estado real en cada plataforma:
Windows y macOS: Maduro. Se integra con wry::WebViewBuilder::build_as_child(&window) pasando el raw-window-handle de winit.
Linux: Maduro en X11 con build_as_child; en Wayland requiere ejecutar la ventana bajo XWayland o incrustar el lienzo sobre una ventana tao/GTK.
Android e iOS: egui (vía eframe + android-activity) e Iced (sobre winit) arrancan en Android e iOS, pero no tienen un widget de WebView nativo. Para mostrar y controlar 10 webviews en Android e iOS debes crear un FrameLayout (Android vía crate jni) y un UIView contenedor (iOS vía crate objc2 / objc2-web-kit) que superponga o alterne los WebView/WKWebView en las coordenadas rectángulares que egui calcule en cada frame (exactamente igual a lo que tendrías que hacer con Slint en móvil).
Rendimiento frente a Slint y Flutter:
RAM del proceso propio: 45–65 MB en escritorio ([Reportado] en https://www.reddit.com/r/rust/comments/1jzawxo/small_footprint_gui_library/, abril de 2025 y https://www.reddit.com/r/rust/comments/jey2uh/built_with_iced_rust_cryptowatch_desktop/), igual o ligeramente inferior a los 74 MB medidos en Slint y casi la mitad de los 122 MB de Flutter.
Tiempo de arranque y binario: Arranque sub-segundo (~0,3–0,7 s, más rápido que Slint según reportes comparativos) y binario de 5–12 MiB con LTO ([Reportado], abril de 2025). Además, en aplicaciones grandes, cambiar una línea de UI en egui tarda ~2–15 s en compilar con 1,1 GB de RAM de rustc, frente a 42–271 s y 6,6–11 GB de RAM en Slint 1.18.1 (https://github.com/slint-ui/slint/issues/4082, [Medido]).
CPU con 10 webviews: ~0 % en reposo/redibujado web porque egui solo repinta su propio lienzo cuando hay eventos de entrada (modo reactivo) y los webviews delegan su composición al SO.
Madurez, licencia y riesgos:
Madurez: Alta en escritorio; media en móvil (teclado virtual IME, safe areas y accesibilidad en Android/iOS son menos maduros en egui/Iced que en Flutter).
Licencia: MIT OR Apache-2.0 (sin obligación de atribución en la app ni restricciones comerciales, a diferencia de Slint).
Riesgo principal: Mantenimiento propio del puente JNI/Objective-C para sincronizar geometría, foco de teclado y subida de archivos de los 10 webviews en Android e iOS.
Opción 2: Makepad 1.0 + Ecosistema Project Robius (Rust GPU Shaders)
Por qué no se consideró / Qué aporta frente a Slint: Es el framework UI en Rust diseñado específicamente para aplicaciones complejas multiplataforma (Windows, macOS, Linux, Android e

iOS) con un motor de shaders propio en GPU, respaldado por Project Robius (que desarrolla en Rust clientes de IA multiplataforma como Moly y clientes Matrix como Robrix corriendo en escritorio, Android e iOS) (https://www.reddit.com/r/rust/comments/1klllldg/makepad_10_rust_ui_framework/, 13 de mayo de 2025; https://hysenlabs.com/projects/makepad-makepad, 23 de septiembre de 2026, [Verificado]).
Fuentes verificables con fecha:
Lanzamiento de Makepad 1.0 y arquitectura de Robius (https://github.com/makepad/makepad, 13 de mayo de 2025, [Verificado]).
Resumen técnico de runtime y Studio (https://hysenlabs.com/projects/makepad-makepad, 23 de septiembre de 2026, [Verificado]).
Estado real en cada plataforma:
Soporta de forma oficial y con empaquetado propio (cargo makepad android / cargo makepad ios) Windows (DirectX 11), macOS (Metal), Linux (OpenGL/X11/Wayland), Android (OpenGL/Vulkan) e iOS (Metal) desde un único código en Rust.
Manejo de 10 webviews nativos: Al igual que Slint y egui, Makepad dibuja toda su interfaz en un lienzo GPU propio (no usa wgpu, sino backends directos de Metal/DX11/GL). Para incrustar 10 webviews nativos requiere usar wry en escritorio y vistas nativas superpuestas (SurfaceView/WebView en Android y WKWebView en iOS) sincronizadas mediante FFI.
Rendimiento frente a Slint y Flutter:
(Inferencia fundamentada en arquitectura) Al no cargar máquina virtual ni motor C++ pesado (como Skia/Impeller en Flutter), su tiempo de arranque (< 0,8 s), tamaño de binario (~6–14 MiB) y RAM del proceso propio (~50–80 MB) están en la misma categoría que Slint y muy por debajo de Flutter.
Madurez, licencia y riesgos:
Madurez: Media (1.0 publicado en mayo de 2025). Documentación aún fragmentada y menor adopción que Slint o egui.
Licencia: MIT OR Apache-2.0.
Riesgo principal: Curva de aprendizaje de su macro DSL en Rust (live_design!), accesibilidad aún en desarrollo y necesidad de construir el adaptador de subvistas de WebView para móvil.
Opción 3: Dioxus Native (Blitz / Vello + wgpu) o Freya (Dioxus + Skia)
Por qué no se consideró (Corrección de descarte): Descartaste Dioxus porque "su interfaz sería un webview más" (lo cual es cierto para dioxus-desktop y dioxus-mobile estándar basados en wry). Sin embargo, el proyecto Dioxus desarrolla Dioxus Native (Blitz), un motor que renderiza HTML/CSS directamente en GPU usando Taffy (layout) y Vello + wgpu (o Skia), sin instanciar ningún WebView para la interfaz (https://github.com/dioxuslabs/blitz, actualizado entre agosto y noviembre de 2026, [Verificado]). De forma paralela existe Freya, que usa el modelo de componentes de Dioxus renderizado con Skia.
Fuentes verificables con fecha:

Estado de Dioxus Native (Blitz) confirmado por su mantenedor (nicoburns): en desarrollo desde diciembre de 2024, con binarios de 10–20 MiB (https://www.libhunt.com/compare-dioxus-vs-slint, 6 de noviembre de 2026; https://www.libhunt.com/compare-dioxus-vs-DioxusLabs--blitz, julio–octubre de 2026, [Verificado]).
Hoja de ruta de Dioxus 0.7 / 0.8 para APIs nativas en Android e iOS (https://github.com/DioxusLabs/dioxus/discussions/5024, 24 de noviembre de 2025, [Verificado]).
Estado real en cada plataforma:
Dioxus Native (Blitz): Funciona en Windows, macOS, Linux, Android e iOS sobre winit + wgpu/vello. (Nota: Freya aún no soporta Android ni iOS de forma oficial según reportes de la comunidad a finales de 2025: https://www.reddit.com/r/rust/comments/1pa4wue/is_there_a_good_rust_alternative_to_flutter/, 30 de noviembre de 2025, [Reportado]).
Manejo de 10 webviews nativos: Aunque Blitz evita que la interfaz de ChatCouncil sea un 11.º webview, Blitz solo renderiza HTML/CSS (no ejecuta el JavaScript complejo de ChatGPT o Claude), por lo que los 10 proveedores siguen requiriendo webviews nativos del SO vía wry en escritorio y JNI/ObjC en móvil.
Rendimiento frente a Slint y Flutter:
Tamaño de binario: 10–20 MiB reportado por el autor de Blitz (https://www.libhunt.com/compare-dioxus-vs-DioxusLabs--blitz, 31 de julio de 2026, [Reportado]), comparable a Slint (11 MiB) y menor que Flutter (25,5 MiB).
RAM del proceso propio: ~60–90 MB (ligeramente superior a Slint por las cachés de pipelines de wgpu/Vello, pero inferior a Flutter).
Madurez, licencia y riesgos:
Madurez: Baja-Media para Dioxus Native (Blitz) en producción móvil (el propio mantenedor señala el 6 de noviembre de 2026 que es menos maduro que los primeros 4 frameworks del ecosistema Rust).
Licencia: MIT OR Apache-2.0.
Opción 4: Kotlin Multiplatform + Compose Multiplatform (CMP) + ComposeNativeWebView (NucleusFramework sobre Tao + Wry) + Núcleo en Rust (UniFFI)
Por qué no se consideró / Qué aporta frente a Slint y Flutter: Si el requisito es un solo código de interfaz para las 5 plataformas que ya tenga resuelta la incrustación de múltiples webviews nativos del SO sin copia de texturas, esta es la única combinación que hoy lo ofrece empaquetado de serie. El núcleo pesado (inyección DOM, orquestación asíncrona de los 10 proveedores, parseo y generación del informe) permanece en Rust expuesto mediante mozilla/uniffi-rs, mientras la interfaz se escribe una sola vez en Compose Multiplatform.
Fuentes verificables con fecha:
ComposeNativeWebView de NucleusFramework: envuelve motores nativos del SO vía Wry (Rust) + Tao + UniFFI en escritorio y vistas nativas en móvil

(https://github.com/NucleusFramework/ComposeNativeWebview, 2026, [Verificado]).
KevinnZou/compose-webview-multiplatform (base de la implementación Android/iOS) (https://github.com/KevinnZou/compose-webview-multiplatform/releases, 29 de agosto de 2025, [Verificado]).
Estado real en cada plataforma (y manejo de 10 webviews simultáneos):
Windows: Usa WebView2 CompositionController + DirectComposition dentro del backend Nucleus Tao NativeView (compose_webview_windows.dll). No renderiza fuera de pantalla a un bitmap: compone directamente por DWM con ~0 % de sobrecarga de CPU y permite dibujar UI de Compose (barras de herramientas, diálogos) encima del WebView.
macOS: Inserta el NSView de WKWebView como subvista de la ventana Tao debajo de la capa Metal de Compose; Compose perfora un recorte transparente (transparent hole) sobre el rectángulo del WebView (https://github.com/NucleusFramework/ComposeNativeWebview, 2026, [Verificado]).
Linux: Reemparenta el widget WebKit2GTK dentro del contenedor de Tao (libcompose_webview_linux.so) sin empaquetar Chromium.
Android e iOS (Múltiples webviews nativos simultáneos): Usa el sistema oficial de interoperabilidad de vistas nativas de Compose (AndroidView con android.webkit.WebView en Android y UIKitView con WKWebView en iOS), incluyendo un puente bidireccional WebViewJsBridge que funciona en las 5 plataformas (https://github.com/NucleusFramework/ComposeNativeWebview, 2026, [Verificado]). A diferencia de wry puro en móvil, en Compose puedes instanciar un bucle o cuadrícula de 10 WebView(state) simultáneos en Android e iOS hoy mismo.
Rendimiento frente a Slint y Flutter:
CPU en redibujado: ~0 % extra en las 5 plataformas (usa composición nativa por hardware/DWM/CoreAnimation en lugar de copia de texturas).
RAM del proceso propio, arranque y tamaño: En Android e iOS es muy ligero (compila a bytecode Android DEX y a código nativo ARM64 con Kotlin/Native en iOS, rondando 15–25 MiB y 60–90 MB de RAM base). Sin embargo, en Escritorio (Windows, macOS, Linux) corre sobre la JVM (Compose Desktop / Skiko), lo que eleva el consumo de RAM del proceso propio a ~110–160 MB, el tamaño del instalador a ~40–65 MiB (con un runtime JRE recortado vía jlink o compilado con GraalVM Native Image) y el arranque en frío en escritorio a ~1,2–2,5 s (Inferencia basada en el runtime JVM/Skiko frente a los 122 MB y 3,35 s medidos en Flutter).
Madurez, licencia y riesgos:
Madurez: Alta en Android, iOS y Escritorio para Compose Multiplatform (JetBrains); Media para el backend de escritorio Nucleus Tao de ComposeNativeWebView (activo en 2026).
Licencia: Apache-2.0 (tanto Compose Multiplatform como compose-webview-multiplatform / ComposeNativeWebView).
Riesgo principal: Mayor consumo de RAM en escritorio debido a la JVM frente a los 74 MB de Slint, y dependencia de la ventana NucleusBackend.Tao en escritorio.
Opción 5: Tauri v2 (multiwebview sobre wry) o Dioxus Desktop/Mobile como Contenedor Ligero
Por qué conviene reevaluar el descarte con números: Descartaste Dioxus/Tauri porque "su interfaz

sería un webview más" (11 webviews en lugar de 10). Sin embargo, en Windows (WebView2), macOS (WKWebView) y Linux (WebKit2GTK), cuando varios webviews comparten el mismo WebContext / CoreWebView2Environment, comparten el mismo proceso navegador, proceso GPU y proceso de red, añadiendo solo un proceso renderizador ligero para la barra de control (~15–25 MB de RAM adicionales, no 122 MB) (https://github.com/tauri-apps/wry/blob/dev/CHANGELOG.md, 2025–2026, [Verificado]).
Fuentes verificables con fecha:
Benchmarks de Tauri v2 en producción: binario de 3–10 MB, memoria en reposo de 40–80 MB y tiempo de arranque < 0,2–1,0 s (https://rustify.rs/articles/rust-tauri-vs-electron-2026, 2 de abril de 2026; https://wrenlearnsrust.com/posts/tauri-2-electron-killer.html, 13 de marzo de 2026, [Reportado]).
Límite de multiwebview en Tauri v2: soportado en escritorio bajo el feature flag unstable, pero no disponible en Android ni iOS (https://github.com/tauri-apps/tauri/issues/11528, 29 de octubre de 2024, [Verificado]).
Veredicto: En escritorio, Tauri v2 con multiwebview arranca igual de rápido que Slint (< 0,8 s), pesa menos (3–10 MiB) y gasta ~50–80 MB en el proceso base, pero queda descartado para Android e iOS si quieres que wry/Tauri gestione los 10 webviews, a menos que en móvil uses un plugin nativo Kotlin/Swift que instancie los 10 WebView/WKWebView hijos fuera de wry.
Opción 6: Opciones Evaluadas y Descartadas por Incumplimiento Técnico en Linux o Móvil (Qt 6 + CXX-Qt, Avalonia UI 11/12, GPUI)
Para evitar que inviertas tiempo probando callejones sin salida, estas tres opciones frecuentes fallan en al menos uno de tus requisitos duros:
Qt 6 (Qt Quick/QML) + CXX-Qt (KDAB) o Qt Bridge for Rust + QtWebView:
Fallo en Linux: Aunque QtWebView usa motores nativos en Windows (WebView2), macOS (WKWebView), Android y iOS (https://doc.qt.io/qt-6/qtwebview-index.html, Qt 6.12, [Verificado]), en Linux QtWebView depende obligatoriamente de QtWebEngine (Chromium empaquetado), disparando el binario en Linux por encima de 150 MiB.
Fallo en Móvil para Qt Bridge for Rust: El nuevo puente oficial de Qt (Qt Bridge for Rust 0.3 Beta, 2026) solo soporta Windows, macOS y Linux; iOS y Android aún no están soportados (https://www.qt.io/development/qt-framework/qt-bridges/rust-gui, 2026, [Verificado]), obligando a usar CXX-Qt de KDAB (MIT/Apache-2.0, https://github.com/KDAB/cxx-qt) junto con la licencia LGPLv3/Comercial de Qt.
.NET 9/10 + Avalonia UI + NativeWebView + Núcleo en Rust:
Fallo de CPU y Licencia en Linux: El control NativeWebView de Avalonia (https://docs.avaloniaui.net/controls/web/nativewebview, 15 de agosto de 2026, [Verificado]) utiliza WPE WebKit en Linux y renderiza fuera de pantalla (offscreen) (https://docs.avaloniaui.net/docs/app-development/embedding-web-content, 15 de agosto de 2026, [Verificado]), lo que reintroduce la penalización de CPU por copia de texturas cuando las 10 páginas redibujan. Además, NativeWebView forma parte del paquete comercial Avalonia Accelerate

(https://docs.avaloniaui.net/tools/faq, 6 de octubre de 2026, [Verificado]).
GPUI (de Zed Industries) o Crux (de Red Badger):
GPUI (adoptado recientemente en reescrituras de escritorio como el cliente experimental de Discord reportado el 5 de noviembre de 2026 en https://yusmpgroup.com/news/discord-electron-rust-desktop-app, [Reportado]) no tiene soporte de producción para Android e iOS con un único código de UI (la propia arquitectura de Discord mantiene UI nativa separada en móvil compartiendo solo el núcleo Rust).
Crux (https://github.com/redbadger/crux, [Verificado]) comparte el núcleo y el estado en Rust (arquitectura Elm sin efectos laterales), pero exige escribir una cáscara de interfaz (Shell) nativa distinta por plataforma (SwiftUI, Jetpack Compose, etc.), incumpliendo tu restricción de un solo código de interfaz.
4.3 Comparativa de Rendimiento y Capacidades frente a Slint y Flutter
(Nota metodológica: las filas de Slint y Flutter reflejan las mediciones exactas del usuario en Windows con 10 instancias de WebView2; las demás filas combinan datos medidos en fuentes primarias/secundarias citadas e inferencia arquitectónica para el mismo escenario de 10 WebViews nativos).
Opción (UI + Núcleo Rust) | Arranque en frío (Escritorio) | Peso del binario base | RAM proceso propio (Escritorio) | CPU extra en redibujado (10 WebViews) | 10 WebViews nativos en Android e iOS | Estado en Linux (X11 y Wayland) | Licencia | Slint + wry (Base medida) | 0,76 s [Medido usuario] | 11 MiB [Medido usuario] | 74 MB [Medido usuario] | ~0 % (HWND hijo / DWM) [Medido usuario] | No en wry; requiere puente JNI/ObjC propio | X11: Sí (build_as_child). Wayland: Requiere XWayland o GTK | GPLv3, Comercial o Royalty-Free (con atribución) |
Flutter + Ventanas nativas (Base medida) | 3,35 s [Medido usuario] | 25,5 MiB [Medido usuario] | 122 MB [Medido usuario] | ~0 % (ventanas separadas) [Medido usuario] | En móvil usa Platform Views (composición híbrida con costo de GPU/hilos) | Ventanas separadas o GTK; sin subvista única limpia en Wayland | BSD-3-Clause |
Flutter + Textura GPU (Base medida) | 3,35 s [Medido usuario] | 25,5 MiB [Medido usuario] | 122 MB [Medido usuario] | +5 % CPU (copia de textura) [Medido usuario] | Sí, pero con sobrecarga de copia de textura/hilos en las 10 vistas | Depende de plugin CEF/WebKit con copia de textura | BSD-3-Clause |
egui (eframe) o Iced + wry + Puente Móvil | ~0,3 – 0,7 s [Reportado] | 5 – 12 MiB [Reportado] | 45 – 65 MB [Reportado] | ~0 % (build_as_child / vista nativa) | Requiere el mismo puente JNI/ObjC (~400 líneas) que Slint | X11: Sí (build_as_child). Wayland: Requiere tao/GTK o XWayland | MIT / Apache-2.0 |
Makepad 1.0 + wry + Puente Móvil | ~0,4 – 0,8 s [Inferencia] | 6 – 14 MiB [Inferencia] | 50 – 80 MB [Inferencia] | ~0 % (build_as_child / vista nativa) | Empaquetado móvil listo (cargo makepad), pero WebViews requieren JNI/ObjC | X11 y Wayland soportados en UI; WebView hijo sujeto a wry | MIT / Apache-2.0 |
Dioxus Native (Blitz) + wry + Puente Móvil | ~0,5 – 0,9 s [Inferencia] | 10 – 20 MiB [Reportado] |

60 – 90 MB [Inferencia] | ~0 % (UI en Vello/wgpu, sin 11.º webview) | En desarrollo (Dioxus 0.8); requiere puente JNI/ObjC para 10 webviews | Igual que egui/Slint (limitado por wry en Wayland) | MIT / Apache-2.0 |
Compose Multiplatform + ComposeNativeWebView + Rust (UniFFI) | ~1,2 – 2,5 s (JVM Desk) / <0,8 s (Móvil) | 35 – 60 MiB (Desk) / 15–25 MiB (Móvil) | 110 – 160 MB (Desk JVM) / 65–90 MB (Móvil) | ~0 % (DirectComposition / NSView / WebKit2GTK) [Verificado] | Sí, incluido de serie (AndroidView + UIKitView + JsBridge) [Verificado] | Sí, incluido de serie (Tao + WebKit2GTK reparentado) [Verificado] | Apache-2.0 |
Tauri v2 (multiwebview) | 0,2 – 0,9 s [Reportado] | 3 – 10 MiB [Reportado] | 50 – 80 MB (+15–25 MB del WebView UI) | ~0 % (wry build_as_child) | No disponible en móvil (Issue #11528) [Verificado] | X11 y Wayland soportados vía tao + gtk::Box/Fixed | MIT / Apache-2.0 |
Avalonia UI 11/12 (NativeWebView) + Rust | ~0,8 – 1,8 s (con NativeAOT) | 20 – 40 MiB | 70 – 110 MB | ~0 % en Win/Mac, Alto en Linux (offscreen) [Verificado] | Sí (NativeControlHost), bajo licencia comercial Accelerate | Renderiza offscreen con WPE WebKit en Linux [Verificado] | MIT (UI) + Comercial (NativeWebView Accelerate) |
Qt 6 (QML) + CXX-Qt + QtWebView | ~0,6 – 1,2 s | 25 – 45 MiB (>150 MiB en Linux) | 65 – 100 MB | ~0 % en Win/Mac/Móvil | Sí (QtWebView usa WebView / WKWebView nativos) [Verificado] | Exige QtWebEngine (Chromium) en Linux [Verificado] | LGPLv3 / GPLv2 / Comercial + MIT (CXX-Qt) |
4.4 El Problema Crítico en Android e iOS: Cómo Sobrevivir a 10 WebViews Simultáneos
Independientemente de si eliges Slint, egui o Compose Multiplatform, abrir 10 aplicaciones web complejas de IA (ChatGPT, Claude, Gemini, DeepSeek, Perplexity, etc.) a la vez en Android e iOS enfrenta dos restricciones severas del sistema operativo que no existen en Windows con 16–64 GB de RAM:
Android (android.webkit.WebView) — Cuello de botella del Proceso Renderizador Único:
(Observación verificada) Según la documentación oficial de ingeniería de Android (https://developer.android.com/topic/performance/memory/guide/webview-memory, 25 de agosto de 2026; https://developer.android.com/develop/ui/views/layout/webapps/manage-webview-memory, 28 de octubre de 2026; https://chromium.googlesource.com/chromium/src/+/HEAD/android_webview/renderer/README.md, [Verificado]), Android divide la memoria del WebView entre el Proceso Host (tu app) —que aloja el motor Browser de Chromium con asignaciones en PartitionAlloc y el pipeline de renderizado GPU integrado en Android HWUI— y un único Proceso Renderizador aislado (SandboxedProcessService0) compartido por todos los WebView de tu aplicación ("Currently, apps just get one renderer process for all WebViews").
Impacto en ChatCouncil y mitigación:
Si mantienes las 10 instancias de android.webkit.WebView adjuntas al árbol de vistas visible al mismo tiempo en Android, HWUI reservará buffers gráficos compartidos en el proceso de tu app para las 10 superficies, y el único proceso sandboxed_process0 acumulará los 10 árboles DOM y

heaps V8.
Mitigación obligatoria en Android: Mantén únicamente 1 o 2 WebView adjuntos al ViewGroup activo (visibles) y los otros 8–9 desacoplados de la vista de hardware o en estado View.GONE (lo que libera los surface buffers de GPU en el proceso Host manteniendo vivo el contexto JS y las conexiones SSE/WebSocket en sandboxed_process0), e implementa WebViewClient.onRenderProcessGone() para reconstruir automáticamente las vistas si el sistema mata el renderizador.
iOS / iPadOS (WKWebView) — Muerte Silenciosa de Procesos por Jetsam (webViewWebContentProcessDidTerminate):
(Observación reportada) Desde iOS 13 en adelante, cada instancia de WKWebView lanza su propio proceso WebContent independiente en el sistema operativo (además de los procesos Networking y GPU) (https://segmentfault.com/a/1190000040652799/en, 8 de septiembre de 2021; https://embrace.io/blog/wkwebview-memory-leaks/, 18 de marzo de 2021, [Reportado]).
Impacto en ChatCouncil y mitigación:
Diez WKWebView cargando SPAs modernas suponen 10 procesos WebContent consumiendo entre 120 MB y 250 MB cada uno (1,2 a 2,5 GB en total). En iOS/iPadOS, antes de matar el proceso principal de tu app, el gestor de memoria Jetsam mata los procesos WebContent de los WKWebView que no están visibles en pantalla o que más memoria consumen, dejando el WKWebView completamente en blanco y disparando el delegado WKNavigationDelegate.webViewWebContentProcessDidTerminate(_:).
Además, en iOS las cookies persistentes (WKWebsiteDataStore.default().httpCookieStore) tienen un retardo de sincronización a disco (Cookies.binarycookies) de 300 ms a 3 s.
Mitigación obligatoria en iOS:
Compartir una única instancia de WKProcessPool y WKWebsiteDataStore.default() entre los 10 WKWebView.
En cuanto un proveedor termine de generar su respuesta y tu script en Rust haya extraído el texto del DOM mediante evaluateJavaScript, serializar el resultado en SQLite/memoria de Rust y, en dispositivos iOS con menos de 8 GB de RAM, navegar los WKWebView inactivos a about:blank (o mantener un pool rotativo de 3–4 WKWebView activos en móvil mientras en escritorio mantienes los 10 simultáneos), e implementar webViewWebContentProcessDidTerminate para recargar la URL de la conversación si Jetsam mata el proceso.
4.5 Diseño Recomendado si Priorizas el Rendimiento Medido de Slint/Rust Puro en las 5 Plataformas
Dado que ya comprobaste que Slint (74 MB RAM, 0,76 s arranque, 11 MiB) supera ampliamente a Flutter (122 MB RAM, 3,35 s arranque, 25,5 MiB) en Windows, tienes dos caminos arquitectónicos sólidos:
Camino A (Máximo rendimiento en las 5 plataformas — Confianza: Alta): Mantener Slint (o cambiar a egui / Makepad si quieres evitar los tiempos de compilación de Slint #4082 y su licencia con atribución #8.2.1) y reemplazar el uso directo de wry por un trait propio en Rust

(MultiWebViewManager) con tres backends ligeros:
Windows y macOS: wry::WebViewBuilder::build_as_child(&window) directo sobre la ventana de Slint/egui (~0 % CPU extra).
Linux: Inicializar la ventana sobre tao + gtk::Fixed con WebViewBuilderExtUnix::build_gtk para que funcione tanto en Wayland como en X11 sin depender de XWayland.
Android e iOS: Un módulo fino de ~350 líneas en Kotlin (vía jni crate) que gestione un FrameLayout con las instancias de android.webkit.WebView y ~350 líneas en Swift/Objective-C (vía objc2-web-kit) que gestione un contenedor UIView con los WKWebView, exponiendo a Rust únicamente 5 funciones: create_or_show_webview(id, rect, url), set_visibility(id, visible), eval_js(id, script), attach_file(id, path) y el callback on_ipc_message(id, json).
Camino B (Menor código nativo manual a costa de ~50–80 MB extra de RAM en Escritorio — Confianza: Moderada-Alta): Usar Compose Multiplatform + ComposeNativeWebView (NucleusFramework) + Núcleo en Rust (UniFFI), que ya trae programados los adaptadores Tao + Wry (DirectComposition, NSView, WebKit2GTK) en escritorio y AndroidView / UIKitView en móvil bajo una sola API declarativa.

5. Fuentes
Fuente | Nivel (Tier) | Fecha | URL | Documentación oficial del crate wry (0.54.0) y código fuente (lib.rs) | Nivel 1 (Primaria) | 15 ene 2026 | https://docs.rs/wry / https://crates.io/crates/wry/0.54.0 |
Repositorio y CHANGELOG.md de tauri-apps/wry | Nivel 1 (Primaria) | Ene–Mar 2026 | https://github.com/tauri-apps/wry |
Issue #1580 (tauri-apps/wry): Falta de soporte multi-ventana/multi-webview en iOS y Android | Nivel 1 (Tracker oficial) | 9 jul 2025 | https://github.com/tauri-apps/wry/issues/1580 |
Issue #1709 (tauri-apps/wry): Limitaciones de WebViewAssetLoader y contexto seguro en Android | Nivel 1 (Tracker oficial) | 31 mar 2026 | https://github.com/tauri-apps/wry/issues/1709 |
Issue #11528 (tauri-apps/tauri): multiwebview limitado a escritorio (unstable), ausente en móvil | Nivel 1 (Tracker oficial) | 29 oct 2024 | https://github.com/tauri-apps/tauri/issues/11528 |
Android Developers: WebView and memory & Manage and diagnose WebView memory | Nivel 1 (Primaria) | 25 ago 2026 / 28 oct 2026 | https://developer.android.com/topic/performance/memory/guide/webview-memory |
Chromium Source: //android_webview/renderer/README.md (proceso renderizador único) | Nivel 1 (Primaria) | Vigente 2026 | https://chromium.googlesource.com/chromium/src/+/HEAD/android_webview/renderer/README.md |
Documentación oficial de Qt 6.12: Módulo Qt WebView y plataformas | Nivel 1 (Primaria) | 2026 | https://doc.qt.io/qt-6/qtwebview-index.html |
Documentación oficial de Qt: Rust GUI with Qt (Qt Bridge for Rust vs CXX-Qt) | Nivel 1 (Primaria) | 2026 | https://www.qt.io/development/qt-framework/qt-bridges/rust-gui |
Documentación oficial de Avalonia UI: NativeWebView, Embedding web content y FAQ | Nivel 1

(Primaria) | 15 ago 2026 / 6 oct 2026 | https://docs.avaloniaui.net/controls/web/nativewebview / https://docs.avaloniaui.net/docs/app-development/embedding-web-content |
Slint Blog oficial: Making Slint Desktop-Ready (Detalles de licencias GPLv3 / Royalty-Free / Comercial) | Nivel 1 (Primaria) | 9 oct 2025 | https://slint.dev/blog/making-slint-desktop-ready |
Issue #4082 (slint-ui/slint): Benchmark empírico de compilación y binario Slint 1.18.1 vs egui e Iced | Nivel 2 (Benchmark reproducible) | Dic 2023 (act. 2026) | https://github.com/slint-ui/slint/issues/4082 |
Repositorio NucleusFramework/ComposeNativeWebview (Compose + Tao + Wry + UniFFI) | Nivel 1 (Primaria) | 2026 | https://github.com/NucleusFramework/ComposeNativeWebview |
Repositorio KevinnZou/compose-webview-multiplatform | Nivel 1 (Primaria) | 29 ago 2025 | https://github.com/KevinnZou/compose-webview-multiplatform/releases |
Documentación y repositorio de egui (emilk/egui) | Nivel 1 (Primaria) | Vigente 2026 | https://github.com/emilk/egui |
Lanzamiento de Makepad 1.0 y resumen de Project Robius | Nivel 1 / 4 | 13 may 2025 / 23 sep 2026 | https://www.reddit.com/r/rust/comments/1kllldg/makepad_10_rust_ui_framework/ / https://hysenlabs.com/projects/makepad-makepad |
Estado de Dioxus Native (Blitz) y roadmap Dioxus 0.8 | Nivel 1 / 4 | 24 nov 2025 / 6 nov 2026 | https://github.com/DioxusLabs/dioxus/discussions/5024 / https://www.libhunt.com/compare-dioxus-vs-slint |
Repositorio oficial de Crux (redbadger/crux) | Nivel 1 (Primaria) | Vigente 2026 | https://github.com/redbadger/crux |
Benchmarks reportados de Tauri v2 (binario, RAM y arranque) | Nivel 4 (Prensa/Blogs técnicos) | 13 mar 2026 / 2 abr 2026 | https://rustify.rs/articles/rust-tauri-vs-electron-2026 / https://wrenlearnsrust.com/posts/tauri-2-electron-killer.html |
Arquitectura multiproceso y límites de memoria de WKWebView en iOS | Nivel 4 (Ingeniería móvil) | 18 mar 2021 / 8 sep 2021 | https://embrace.io/blog/wkwebview-memory-leaks/ / https://segmentfault.com/a/1190000040652799/en |

6. Conflictos, Vacíos y Límites de Verificación
   Ausencia de un benchmark idéntico de 10 WebViews en móvil: Tus mediciones en Windows con 10 instancias de WebView2 son empíricamente superiores a cualquier benchmark genérico de "Hello World" publicado en la web. No existen mediciones públicas que ejecuten exactamente 10 instancias de IA en paralelo comparando Slint, egui, Makepad y Compose Multiplatform en el mismo dispositivo Android e iOS; las cifras de RAM y arranque para esas alternativas provienen de aplicaciones de referencia sin los 10 webviews activos (https://gpui-kit.com/docs/comparison, [Verificado]).
   Varianza en el tamaño de binario de Slint según backend y plataforma: Mientras en tu medición en Windows Slint pesó 11 MiB, el estudio de GPUI Kit midió 19,81 MiB para Slint 1.18.1 en Linux x86-64 (cargo build --release con strip), y el caso de estudio industrial en el Issue #4082 (con 111 archivos .slint) alcanzó 87 MB de binario antes de optimizaciones de unidades de generación de

código. Esto indica que el binario de Slint crece linealmente con el número de componentes declarados en .slint debido a la monomorfización del código generado.
Lo que no se verificó directamente: No se ejecutó en vivo una prueba de estrés de 10 sesiones autenticadas simultáneas en iOS 26/27 ni en Android 15/16 para determinar el umbral exacto de RAM en el que Jetsam (iOS) o el ActivityManager / SandboxedProcessService0 (Android) matan el primer proceso renderizador.

7. Preguntas Abiertas y Próximos Pasos Recomendados
   Prueba de fuego de 48 horas en Android e iOS antes de elegir el framework UI: Construye un prototipo mínimo en Android (Kotlin) e iOS (Swift) que instancie 10 WebView / WKWebView reales cargando los 10 proveedores de IA de ChatCouncil y mide:
   En Android: el TOTAL PSS del proceso com.google.android.webview:sandboxed_process0 mediante adb shell dumpsys meminfo <PID> (https://developer.android.com/topic/performance/memory/guide/webview-memory) cuando los 10 proveedores reciben un prompt a la vez.
   En iOS: cuántos procesos WebContent sobreviven simultáneamente antes de que se dispare webViewWebContentProcessDidTerminate en un iPhone/iPad estándar.
   Validación en Linux Wayland: Prueba tu prototipo actual de Slint + wry en una sesión Linux Wayland pura (desactivando XWayland). Si build_as_child falla o desposiciona las vistas, evalúa inicializar la ventana con tao + gtk::Fixed (WebViewBuilderExtUnix::build_gtk).
   Decisión sobre la capa UI en Rust: Si tu interfaz de control en ChatCouncil crecerá poco (menos de 15 pantallas), Slint es excelente y ya lo tienes medido; si planeas decenas de vistas complejas o quieres evitar la licencia con atribución y los tiempos de compilación del generador .slint, haz una prueba rápida con egui (eframe) + wry, que reutiliza exactamente tu mismo código de wry en escritorio.
   A continuación puedes explorar y filtrar interactivamente las métricas medidas y estimadas de cada arquitectura (RAM del proceso propio, tamaño del binario, tiempo de arranque y sobrecarga de CPU en redibujado):
   No se pudo cargar el elemento visual interactivoPresiona para volver a intentarlo
   Para profundizar en la implementación técnica de ChatCouncil:
   Diseñar la arquitectura del puente JNI y Objective-C para 10 WebViews
   Estrategia contra OOM y Jetsam con 10 WebViews en Android e iOS
   Resolver el soporte de múltiples WebViews con wry en Linux Wayland

---

Texto capturado tal cual de la interfaz del proveedor, sin corregir ni recortar.
Este documento es la respuesta a la pregunta, no la evaluacion que este proveedor hizo de las respuestas de los demas.